

```

long syscall(long number, ...);

/* example program to invoke write(2) system call via
syscall(2)*/

#include <unistd.h>
#include <sys/syscall.h>

int main ()
{
    syscall (SYS_write, 1, "Hello syscall", 13);
    return 0;
}

```

Checking the return value

In x86-64 architecture, before resuming user space, the Linux kernel puts the system call handler return value in the rax register. Hence the glibc system call wrapper function (after executing the `syscall` instruction) reads the system call return value from the rax register.

In case of success the kernel sets a positive value in the rax register, while in case of error it sets a negative value. This negative value is actually one of the negated values of 'errno' constant defined in `/usr/include/asm-generic/errno-base.h`. In case of a positive value in the rax register, glibc wrapper function gives 0 (success) as return value of system call to the user application program. However, in case of negative value, glibc wrapper gives -1 (failure) as return value to the user application program and sets 'errno' by negating the value in the rax register; this means it will be a positive value defined as one of the errno constants in `/usr/include/asm-generic/errno-base.h`. Given below are some of the code snippets and comments from the glibc code related with handling of the return value from the system call. Please check the code comments marked in red.

`sysdeps/unix/sysv/linux/x86_64/sysdep.h`

```

/* Linux uses a negative return value to indicate syscall
errors, unlike most Unices, which use the condition codes'
carry flag.

Since version 2.1 the return value of a system call might be
negative even if the call succeeded. E.g., the 'lseek'
system call might return a large offset. Therefore, we must
not anymore test for < 0, but test for a real error by making
sure the value in %eax is a real error number. Linus said
he will make sure the no syscall returns a value in -1 ..
-4095 as a valid result so we can safely test with -4095.
*/

# undef INLINE_SYSCALL
# define INLINE_SYSCALL(name, nr, args...) \

```

```

({

    /* Further INTERNAL_SYSCALL macro is being invoked*/

    unsigned long int resultvar = INTERNAL_SYSCALL (name, ,
nr, args);

    /* This macro check the range of return value */
    if (__glibc_unlikely (INTERNAL_SYSCALL_ERROR_P
(resultvar, )))
    {
        /* set errno by negating the return value */
        __set_errno (INTERNAL_SYSCALL_ERRNO (resultvar,
));
        resultvar = (unsigned long int) -1
    }
    (long int) resultvar; })

# undef INTERNAL_SYSCALL_ERROR_P
# define INTERNAL_SYSCALL_ERROR_P(val, err) \
    /* Range of error value is -1 to -4095 */
    ((unsigned long int) (long int) (val) >= -4095L)

# undef INTERNAL_SYSCALL_ERRNO
/*negating the value */
# define INTERNAL_SYSCALL_ERRNO(val, err)    (-(val))

cat /usr/include/asm-generic/errno-base

#ifdef _ASM_GENERIC_ERRNO_BASE_H
#define _ASM_GENERIC_ERRNO_BASE_H

#define EPERM            1    /* Operation not permitted */
#define ENOENT            2    /* No such file or directory */
#define ESRCH            3    /* No such process */
#define EINTR            4    /* Interrupted system call */
#define EIO              5    /* I/O error */
#define ENXIO            6    /* No such device or address */
#define E2BIG            7    /* Argument list too long */
#define ENOEXEC          8    /* Exec format error */
#define EBADF            9    /* Bad file number */
#define ECHILD           10   /* No child processes */
#define EAGAIN           11   /* Try again */
#define ENOMEM           12   /* Out of memory */
#define EACCES           13   /* Permission denied */
#define EFAULT           14   /* Bad address */
#define ENOTBLK          15   /* Block device required */
#define EBUSY            16   /* Device or resource busy */
#define EEXIST           17   /* File exists */
#define EXDEV            18   /* Cross-device link */
#define ENODEV           19   /* No such device */
#define ENOTDIR          20   /* Not a directory */

```